

Lesson 5: Building a Simple RAG System

Now it's time to put everything together! In this lesson, you'll build a complete RAG system step by step.

What We're Building

A simple but functional RAG system that: - ■ Loads and processes documents - ■ Chunks text intelligently - ■ Creates and stores embeddings - ■ Retrieves relevant context - ■ Generates answers with source citations - ■ Provides a command-line interface

Project Structure

```
simple_rag/
├── rag_system.py      # Main RAG system class
├── document_loader.py # Load various document types
├── chunker.py         # Text chunking utilities
├── cli.py             # Command-line interface
├── config.py          # Configuration
├── data/              # Sample documents
│   ├── sample1.txt
│   └── sample2.txt
```

Step 1: Configuration

Let's start with a configuration file:

```
# config.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    # LLM Settings
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
```

```

LLM_MODEL = "gpt-3.5-turbo"
LLM_TEMPERATURE = 0.7
MAX_TOKENS = 500

# Embedding Settings
EMBEDDING_MODEL = "all-MiniLM-L6-v2" # Free, local model

# Chunking Settings
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50

# Retrieval Settings
TOP_K_RESULTS = 3

# Vector DB Settings
COLLECTION_NAME = "documents"
PERSIST_DIRECTORY = "./chroma_db"

```

Step 2: Document Loader

Load different types of documents:

```

# document_loader.py
import os
from typing import List, Dict
from pathlib import Path

class DocumentLoader:
    """Load documents from various sources"""

    @staticmethod
    def load_txt(file_path: str) -> Dict:
        """Load text file"""
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()

        return {
            'content': content,
            'metadata': {
                'source': file_path,
                'type': 'txt'
            }
        }

    @staticmethod
    def load_directory(directory: str, extensions: List[str] = ['.txt']) -> List[Dict]:
        """Load all documents from directory"""
        documents = []

        for file_path in Path(directory).rglob('*'):
            if file_path.suffix in extensions and file_path.is_file():
                try:
                    doc = DocumentLoader.load_txt(str(file_path))

```

```

        documents.append(doc)
        print(f"Loaded: {file_path}")
    except Exception as e:
        print(f"Error loading {file_path}: {e}")

    return documents

@staticmethod
def load_from_text(text: str, metadata: Dict = None) -> Dict:
    """Load from raw text string"""
    return {
        'content': text,
        'metadata': metadata or {'source': 'direct_input', 'type': 'text'}
    }

```

Step 3: Text Chunking

Implement smart text chunking:

```

# chunker.py
from typing import List, Dict
import re

class TextChunker:
    """Chunk text into smaller pieces"""

    def __init__(self, chunk_size: int = 500, overlap: int = 50):
        self.chunk_size = chunk_size
        self.overlap = overlap

    def chunk_text(self, text: str, metadata: Dict = None) -> List[Dict]:
        """
        Split text into overlapping chunks
        Returns list of dicts with 'content' and 'metadata'
        """
        # Clean text
        text = self._clean_text(text)

        # Split into chunks
        chunks = []
        start = 0
        chunk_id = 0

        while start < len(text):
            # Get chunk
            end = start + self.chunk_size
            chunk_text = text[start:end]

            # Try to end at sentence boundary
            if end < len(text):
                # Look for sentence end
                last_period = chunk_text.rfind('.')
                last_newline = chunk_text.rfind('\n')

```

```

        boundary = max(last_period, last_newline)
        if boundary > self.chunk_size * 0.5: # At least 50% through
            chunk_text = chunk_text[:boundary + 1]
            end = start + boundary + 1

        # Create chunk
        chunk_metadata = metadata.copy() if metadata else {}
        chunk_metadata['chunk_id'] = chunk_id
        chunk_metadata['start'] = start
        chunk_metadata['end'] = end

        chunks.append({
            'content': chunk_text.strip(),
            'metadata': chunk_metadata
        })

        # Move to next chunk (with overlap)
        start = end - self.overlap
        chunk_id += 1

    return chunks

def _clean_text(self, text: str) -> str:
    """Clean and normalize text"""
    # Remove excessive whitespace
    text = re.sub(r'\s+', ' ', text)
    # Remove excessive newlines
    text = re.sub(r'\n{3,}', '\n\n', text)
    return text.strip()

def chunk_documents(self, documents: List[Dict]) -> List[Dict]:
    """Chunk multiple documents"""
    all_chunks = []

    for doc in documents:
        chunks = self.chunk_text(doc['content'], doc.get('metadata'))
        all_chunks.extend(chunks)

    return all_chunks

```

Step 4: The RAG System

Now the main RAG system:

```

# rag_system.py
from typing import List, Dict, Optional
import chromadb
from chromadb.config import Settings
from sentence_transformers import SentenceTransformer
from openai import OpenAI
from config import Config
from chunker import TextChunker

```

```

from document_loader import DocumentLoader

class RAGSystem:
    """Complete RAG system"""

    def __init__(self, config: Config = Config()):
        self.config = config

        # Initialize components
        self.chunker = TextChunker(
            chunk_size=config.CHUNK_SIZE,
            overlap=config.CHUNK_OVERLAP
        )

        # Initialize embedding model
        print(f"Loading embedding model: {config.EMBEDDING_MODEL}...")
        self.embedding_model = SentenceTransformer(config.EMBEDDING_MODEL)

        # Initialize vector database
        self.chroma_client = chromadb.Client(Settings(
            persist_directory=config.PERSIST_DIRECTORY,
            anonymized_telemetry=False
        ))

        # Create or get collection
        self.collection = self.chroma_client.get_or_create_collection(
            name=config.COLLECTION_NAME,
            metadata={"description": "RAG document collection"}
        )

        # Initialize LLM (if API key provided)
        if config.OPENAI_API_KEY:
            self.llm = OpenAI(api_key=config.OPENAI_API_KEY)
        else:
            self.llm = None
            print("Warning: No OpenAI API key. Generation will be limited.")

    def add_documents(self, documents: List[Dict]) -> int:
        """
        Add documents to the system
        Returns number of chunks added
        """
        print(f"Processing {len(documents)} documents...")

        # Chunk documents
        chunks = self.chunker.chunk_documents(documents)
        print(f"Created {len(chunks)} chunks")

        # Prepare for insertion
        ids = [f"chunk_{i}" for i in range(len(chunks))]
        contents = [chunk['content'] for chunk in chunks]
        metadatas = [chunk['metadata'] for chunk in chunks]

        # Add to collection
        self.collection.add(
            ids=ids,
            documents=contents,
            metadatas=metadatas
        )

```

```

        print(f"Added {len(chunks)} chunks to vector database")
        return len(chunks)

    def add_directory(self, directory: str) -> int:
        """Load and add all documents from directory"""
        documents = DocumentLoader.load_directory(directory)
        return self.add_documents(documents)

    def retrieve(self, query: str, n_results: Optional[int] = None) -> List[Dict]:
        """
        Retrieve relevant chunks for query
        Returns list of dicts with 'content', 'metadata', 'distance'
        """
        if n_results is None:
            n_results = self.config.TOP_K_RESULTS

        results = self.collection.query(
            query_texts=[query],
            n_results=n_results
        )

        # Format results
        retrieved = []
        for i in range(len(results['documents'][0])):
            retrieved.append({
                'content': results['documents'][0][i],
                'metadata': results['metadatas'][0][i],
                'distance': results['distances'][0][i]
            })

        return retrieved

    def generate_answer(self, query: str, context: List[Dict]) -> str:
        """Generate answer using LLM"""
        if not self.llm:
            return "Error: No LLM configured. Please set OPENAI_API_KEY."

        # Format context
        context_str = "\n\n".join([
            f"[{i+1}] {chunk['content']}\n(Source: {chunk['metadata'].get('source', 'unknown')}"
            for i, chunk in enumerate(context)
        ])

        # Create prompt
        prompt = f"""Answer the question based on the context below.
        If you cannot answer based on the context, say so clearly.
        Cite the source numbers you reference in your answer.

        Context:
        {context_str}

        Question: {query}

        Answer: """

        try:
            # Generate response
            response = self.llm.chat.completions.create(
                model=self.config.LLM_MODEL,
                messages=[

```

```

        {"role": "system", "content": "You are a helpful assistant that answers qu
        {"role": "user", "content": prompt}
    ],
    temperature=self.config.LLM_TEMPERATURE,
    max_tokens=self.config.MAX_TOKENS
)

    return response.choices[0].message.content

except Exception as e:
    return f"Error generating response: {str(e)}"

def query(self, question: str, return_sources: bool = True) -> Dict:
    """
    Complete RAG pipeline: retrieve + generate
    """
    # Retrieve relevant chunks
    retrieved = self.retrieve(question)

    # Generate answer
    answer = self.generate_answer(question, retrieved)

    # Format response
    result = {
        'question': question,
        'answer': answer,
    }

    if return_sources:
        result['sources'] = retrieved

    return result

def get_stats(self) -> Dict:
    """Get statistics about the system"""
    count = self.collection.count()
    return {
        'total_chunks': count,
        'collection_name': self.config.COLLECTION_NAME,
        'embedding_model': self.config.EMBEDDING_MODEL,
        'llm_model': self.config.LLM_MODEL if self.llm else 'None'
    }

```

Step 5: Command-Line Interface

Create a simple CLI:

```

# cli.py
import argparse
from pathlib import Path
from rag_system import RAGSystem
from document_loader import DocumentLoader
from config import Config

```

```

def print_result(result: dict):
    """Pretty print query result"""
    print("\n" + "="*80)
    print("QUESTION:")
    print(result['question'])
    print("\n" + "-"*80)
    print("ANSWER:")
    print(result['answer'])

    if 'sources' in result:
        print("\n" + "-"*80)
        print("SOURCES:")
        for i, source in enumerate(result['sources'], 1):
            print(f"\n[{i}] (similarity: {1 - source['distance']:.3f})")
            print(f"Source: {source['metadata'].get('source', 'unknown')}")
            print(f"Content: {source['content'][:200]}...")

def main():
    parser = argparse.ArgumentParser(description="Simple RAG System")
    subparsers = parser.add_subparsers(dest='command', help='Commands')

    # Add documents command
    add_parser = subparsers.add_parser('add', help='Add documents')
    add_parser.add_argument('path', help='File or directory path')

    # Query command
    query_parser = subparsers.add_parser('query', help='Query the system')
    query_parser.add_argument('question', help='Question to ask')

    # Stats command
    subparsers.add_parser('stats', help='Show system statistics')

    # Interactive command
    subparsers.add_parser('interactive', help='Interactive mode')

    args = parser.parse_args()

    # Initialize system
    config = Config()
    rag = RAGSystem(config)

    if args.command == 'add':
        path = Path(args.path)
        if path.is_file():
            doc = DocumentLoader.load_txt(str(path))
            count = rag.add_documents([doc])
        elif path.is_dir():
            count = rag.add_directory(str(path))
        else:
            print(f"Error: {path} is not a valid file or directory")
            return

        print(f"\n✓ Successfully added {count} chunks")

    elif args.command == 'query':
        result = rag.query(args.question)
        print_result(result)

    elif args.command == 'stats':

```

```

        stats = rag.get_stats()
        print("\nSystem Statistics:")
        print(f"  Total chunks: {stats['total_chunks']}")
        print(f"  Collection: {stats['collection_name']}")
        print(f"  Embedding model: {stats['embedding_model']}")
        print(f"  LLM model: {stats['llm_model']}")

    elif args.command == 'interactive':
        print("\nInteractive RAG System")
        print("Type 'quit' to exit\n")

        while True:
            try:
                question = input("Question: ").strip()
                if question.lower() in ['quit', 'exit', 'q']:
                    break

                if not question:
                    continue

                result = rag.query(question)
                print_result(result)
                print("\n")

            except KeyboardInterrupt:
                break

        print("\nGoodbye!")

    else:
        parser.print_help()

if __name__ == "__main__":
    main()

```

Step 6: Sample Data

Create sample documents to test with:

```

# Create data/sample1.txt
"""
Retrieval-Augmented Generation (RAG)

RAG is a technique in natural language processing that combines
retrieval systems with generative models. It was introduced by
Meta AI researchers in 2020.

The key innovation of RAG is that it allows language models to
access external knowledge bases during generation. This helps
address several limitations of standard language models:

1. Knowledge Cutoff: Standard LLMs only know information from
their training data

```

2. Hallucinations: LLMs sometimes make up convincing but false information
3. Source Attribution: RAG systems can cite their sources

The RAG process has three main steps:

- Indexing: Documents are embedded and stored in a vector database
- Retrieval: Relevant documents are found using similarity search
- Generation: An LLM generates a response using the retrieved context

```
"""  
# Create data/sample2.txt  
"""
```

Vector Embeddings

Vector embeddings are numerical representations of text that capture semantic meaning. They are the foundation of modern RAG systems.

Popular embedding models include:

- OpenAI's text-embedding-ada-002 (1536 dimensions)
- Sentence Transformers (384-768 dimensions)
- Google's Universal Sentence Encoder

Embeddings enable semantic search, which finds similar concepts even when different words are used. For example, "car" and "automobile" would have very similar embeddings.

The quality of embeddings directly impacts RAG performance. Better embeddings lead to better retrieval, which leads to better answers.

```
"""
```

Step 7: Usage Guide

Installation

```
# Install dependencies  
pip install chromadb sentence-transformers openai python-dotenv  
  
# Set up environment  
echo "OPENAI_API_KEY=your-key-here" > .env
```

Basic Usage

```
# Add documents  
python cli.py add data/  
  
# Check stats  
python cli.py stats
```

```
# Ask a question
python cli.py query "What is RAG?"

# Interactive mode
python cli.py interactive
```

Python API Usage

```
from rag_system import RAGSystem
from document_loader import DocumentLoader

# Initialize
rag = RAGSystem()

# Add documents
docs = DocumentLoader.load_directory("data/")
rag.add_documents(docs)

# Query
result = rag.query("What are vector embeddings?")
print(result['answer'])
```

Testing Your System

Test 1: Basic Retrieval

```
# Test that retrieval works
retrieved = rag.retrieve("What is RAG?")
for chunk in retrieved:
    print(f"Score: {1 - chunk['distance']:.3f}")
    print(f"Content: {chunk['content'][:100]}...\n")
```

Test 2: Question Answering

```
# Test end-to-end QA
questions = [
    "What is RAG?",
    "What are vector embeddings?",
    "What is semantic search?",
]
```

```
for q in questions:
    result = rag.query(q)
    print(f"Q: {q}")
    print(f"A: {result['answer']}\n")
```

Test 3: Source Attribution

```
# Verify sources are cited
result = rag.query("How many dimensions do embeddings have?")
print(result['answer'])
print(f"\nSources used: {len(result['sources'])}")
```

Evaluation Metrics

1. Retrieval Quality

```
def evaluate_retrieval(rag, test_cases):
    """
    test_cases = [
        ("question", "expected_doc_content"),
        ...
    ]
    """
    scores = []
    for question, expected in test_cases:
        results = rag.retrieve(question, n_results=3)
        # Check if expected content is in top results
        found = any(expected.lower() in r['content'].lower() for r in results)
        scores.append(1 if found else 0)

    return sum(scores) / len(scores)
```

2. Answer Quality (Manual)

Check if answers: - ✓ Are accurate based on context - ✓ Cite sources appropriately - ✓ Admit when they don't know - ✓ Are concise and relevant

Common Issues and Solutions

Issue: "No results found"

Cause: No documents added or query too different from content

Solution: Add documents first, try rephrasing query

Issue: "Poor answer quality"

Cause: Irrelevant chunks retrieved

Solution: Adjust chunk size, try different embedding model, add more context

Issue: "Slow queries"

Cause: Large number of documents

Solution: Use persistent ChromaDB, add metadata filters, use better hardware

Issue: "API errors"

Cause: Invalid or missing OpenAI API key

Solution: Check .env file, verify key is valid

Extending the System

Ideas for improvements: 1. **Add more document types**: PDFs, Word docs, URLs 2. **Implement re-ranking**: Re-order results with cross-encoder 3. **Add conversation history**: Multi-turn conversations 4. **Hybrid search**: Combine semantic + keyword search 5. **Better chunking**: Semantic or recursive chunking 6. **Streaming responses**: Show answers as they're generated

What You've Learned

- Built a complete RAG system from scratch
- Implemented document loading and chunking
- Created retrieval and generation pipelines
- Built a CLI for easy interaction
- Tested and evaluated the system

Next Steps

In [Lesson 6: Advanced RAG Techniques](#), you'll learn: - Advanced chunking strategies - Re-ranking for better results - Hybrid search techniques - Query optimization - Production considerations

Practice Exercises

1. **Add your own documents**: Load your personal notes or documents
 2. **Customize chunking**: Experiment with different chunk sizes
 3. **Improve prompts**: Modify the generation prompt for better answers
 4. **Add features**: Implement conversation history or document filtering
-

